

Backend Engineering Guide

1. Core Backend Skills Overview

A well-rounded backend developer is expected to be fluent in at least one server-side language (such as Node.js, Java, Python, or Go), comfortable designing and consuming APIs, and capable of modeling and querying relational and/or NoSQL databases. Beyond raw coding ability, strong backend engineers understand how their code behaves under concurrent load, how to reason about failure modes, and how to keep a system observable in production.

Version control fluency (Git branching, pull requests, code review), automated testing (unit and integration tests), and basic Linux/command-line proficiency are considered table stakes rather than differentiators. What separates a mid-level from a senior backend engineer is usually system design judgment: knowing when to introduce a queue, when a cache will actually help, and when added complexity is not worth the operational cost.

Common skill gaps recruiters flag on backend resumes include: no evidence of designing (not just consuming) APIs, no mention of database indexing or query optimization, no exposure to authentication/authorization patterns, and no experience with production monitoring, logging, or incident response.

Backend Engineering Guide

2. API Design Best Practices (REST and GraphQL)

REST APIs should model resources as nouns (e.g. `/applications`, `/jobs/:id`) and use HTTP methods to express the action (GET to read, POST to create, PATCH to partially update, DELETE to remove). Status codes should be used meaningfully: 200 for success, 201 for created, 400 for a client error like bad input, 401 for missing authentication, 403 for authenticated-but-forbidden, 404 for a missing resource, and 500 for an unexpected server failure.

Version your API from day one (either in the URL path like `/api/v1` or via a header) so that breaking changes do not silently break existing clients. Validate and sanitize all request input at the boundary of the system, never trusting client-supplied data, and return clear, structured error messages that a frontend can act on programmatically rather than only a human-readable string.

GraphQL is a strong fit when clients need to fetch deeply nested, variably-shaped data in a single round trip (for example, a dashboard that needs a job, its applications, and each candidate's profile at once), but it adds real complexity around caching, rate-limiting individual field resolvers, and query depth/complexity limits to prevent abusive queries. A REST API is usually simpler to build, cache, and reason about for CRUD-style resources like the ones in a typical job board or applicant tracking system.

Backend Engineering Guide

3. Database Design and Query Optimization

Normalize relational schemas to avoid duplicated data and update anomalies, but be willing to selectively denormalize for read-heavy paths where join costs are measurably too high. Every foreign key column and every column used in a WHERE, JOIN, or ORDER BY clause on a large table is a candidate for an index — without one, the database falls back to a full table scan that gets slower as the table grows.

Use EXPLAIN (or EXPLAIN ANALYZE in PostgreSQL) to inspect a slow query's actual execution plan rather than guessing; look specifically for sequential scans on large tables where an index scan would be expected. Batch writes and use connection pooling (such as pg-pool) rather than opening a new database connection per request, since connection setup is comparatively expensive and connection limits are a common production bottleneck.

Be deliberate about transaction boundaries: wrap multi-statement writes that must succeed or fail together in an explicit transaction, and understand your database's default isolation level well enough to reason about race conditions like lost updates or double-processing on concurrent requests.

Backend Engineering Guide

4. Authentication and Security

Never store passwords in plain text; always hash them with a slow, purpose-built algorithm such as bcrypt or argon2, never a fast general-purpose hash like MD5 or SHA-256 alone. For session state, prefer signed, httpOnly, secure cookies over storing tokens in localStorage, since localStorage is readable by any JavaScript running on the page and is therefore vulnerable to XSS-based token theft.

JSON Web Tokens (JWTs) are a common way to represent an authenticated session or to federate identity from an external provider, but they must be signed with a strong secret or key, given a reasonable expiration, and verified on every request — never trust an unverified JWT's claims. Role-based access control (RBAC) should be enforced on the server for every sensitive route, never solely in frontend UI logic, since a client can always be modified to hide or bypass frontend checks.

Apply the OWASP Top 10 as a baseline security checklist: parameterize all SQL queries to prevent injection, validate and escape any user input that is rendered back to a browser to prevent XSS, enforce HTTPS everywhere, and set a sane CORS policy that allows only the origins your application actually needs.

Backend Engineering Guide

5. Scalability, Caching, and Message Queues

Horizontal scaling — running multiple stateless instances of your application behind a load balancer — is the standard way to handle increasing traffic, which requires that your application not keep session state in local process memory (use a shared store like Redis or a database instead). Identify your system's actual bottleneck (CPU, memory, database I/O, or network) before choosing a scaling strategy; adding more application servers does nothing if the database is the constraint.

Caching frequently-read, rarely-changed data (such as job listings or reference data) in an in-memory store like Redis can dramatically reduce database load and latency, but requires a clear invalidation strategy — a cache that serves stale data after an update is often worse than no cache at all. A simple time-to-live (TTL) expiration is often good enough; more complex write-through or cache-aside patterns are only worth the complexity when staleness genuinely matters.

Message queues (such as RabbitMQ, Amazon SQS, or Kafka) decouple producers from consumers and let you smooth out traffic spikes, retry failed work, and process expensive tasks (sending emails, generating reports, resizing images) asynchronously instead of blocking the request/response cycle. Understanding at-least-once versus exactly-once delivery semantics, and designing consumers to be idempotent, is essential to using queues correctly in production.

Backend Engineering Guide

6. Testing and CI/CD for Backend Systems

A healthy backend test suite has a mix of fast unit tests for pure logic, integration tests that exercise real database queries against a test database, and a smaller number of end-to-end tests that verify complete request/response flows through the actual API. Favor integration tests over heavily mocked unit tests for anything touching the database — mocks can pass while the real query or migration is broken.

Continuous integration should run the full test suite, type-checking, and linting on every pull request before it can be merged, catching regressions before they reach production. Continuous deployment pipelines should include automated database migrations, health checks after deploy, and an easy rollback path if the health check fails.

Structured logging (JSON logs with request IDs) and application performance monitoring (tracking latency percentiles, error rates, and throughput per endpoint) are what make production incidents diagnosable quickly; a backend engineer who can point to concrete monitoring and incident-response experience stands out strongly from one who has only worked in local development.